

Gerenciamento de Memória

O gerenciamento de memória é um aspecto fundamental dos sistemas operacionais. Aqui estão alguns pontos principais que vamos abordar:

- Como o SO abstrai e gerencia os acessos à memória feitos pelos programas.
- Como é possível para múltiplos programas compartilharem com segurança a mesma memória física.
- Como o SO fornece a ilusão de ter mais memória do que a disponível fisicamente para executar programas.

Desafio com Segmentação

Com a **segmentação**, o SO divide a memória física em partes de tamanho variável.

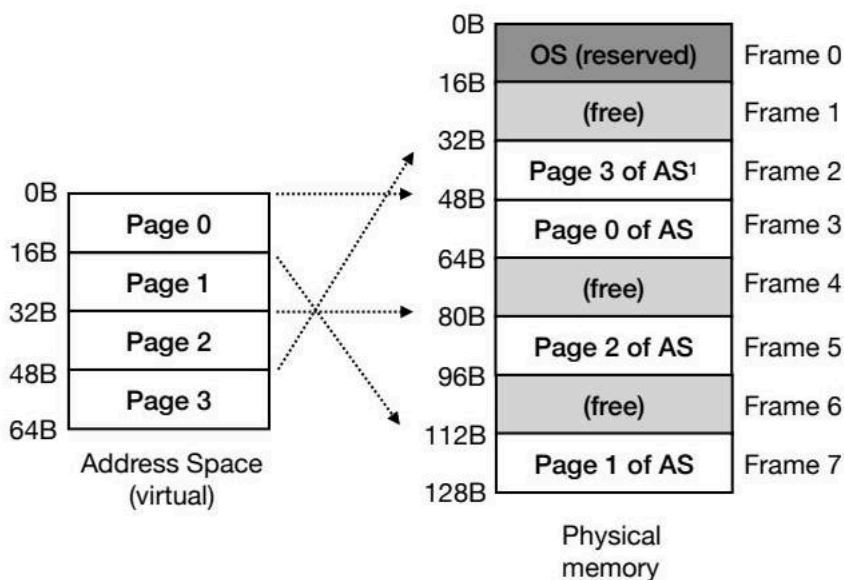
Isso pode levar à **fragmentação externa**, o que torna a alocação de memória desafiadora ao longo do tempo.

Como Evitar a Fragmentação Externa?

Precisamos de novas técnicas que sejam eficientes em termos de desempenho e espaço.

Páginas e Frames (Paging)

Com o **paging**, o espaço de endereço virtual é dividido em partes de tamanho fixo chamadas **páginas virtuais**. A memória física é dividida em partes de tamanho fixo, chamadas **page frames**. Cada frame físico contém uma única página virtual para evitar fragmentação.



A imagem ilustra como páginas virtuais (Page 0 a Page 3) são mapeadas para frames físicos (Frame 3, Frame 7, Frame 5, Frame 2) na memória, demonstrando o conceito de paginação.

Vantagens do Paging

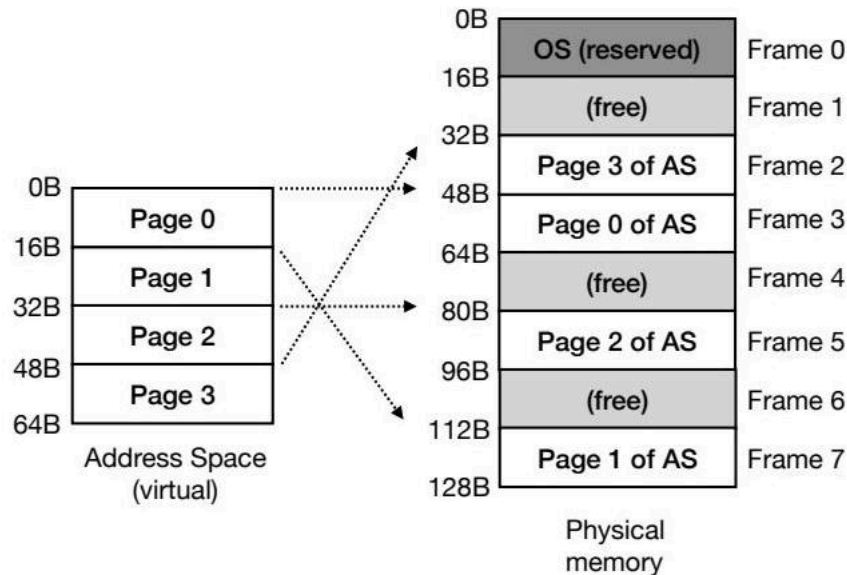
- **Simplicidade** no gerenciamento do espaço livre: a lista de frames livres facilita a alocação.
- **Flexibilidade** no suporte à abstração do espaço de endereço: independe de como os processos usam o espaço de endereço.

Tabela de Páginas e Tradução de Endereços

A **tradução de endereços** ainda precisa ser feita. Páginas virtuais são mapeadas para a memória física, ajudando o SO a construir a abstração de memória privada do processo.

Cada endereço de página virtual deve ser traduzido para o endereço de frame físico correspondente. Uma **tabela de páginas** é usada para registrar onde cada página está localizada na memória física.

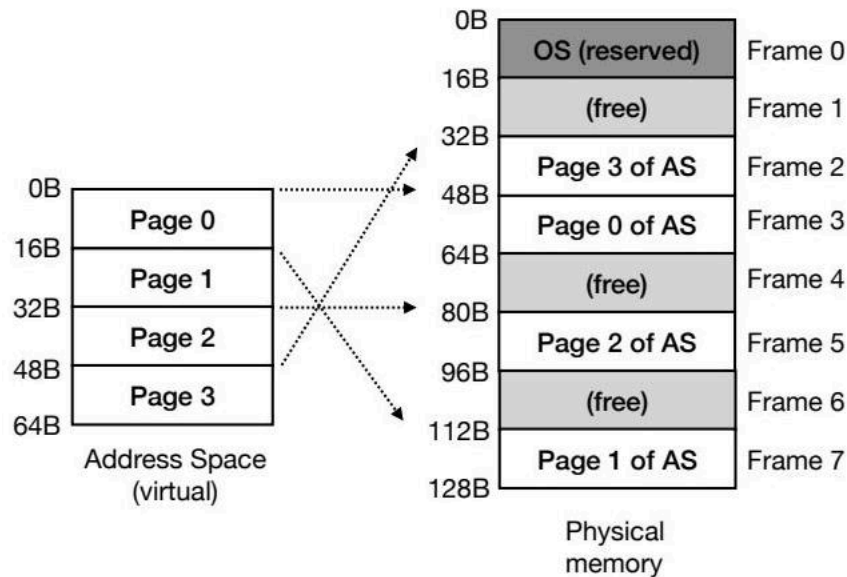
A tabela de páginas é uma estrutura de dados por processo, ou seja, cada processo tem sua própria tabela de páginas.



A imagem mostra como as páginas virtuais (Page 0 a Page 3) são mapeadas para os frames físicos correspondentes (Frame 3, Frame 7, Frame 5, Frame 2), destacando a alocação de páginas virtuais para frames físicos disponíveis.

Exemplo de Tradução de Endereços

Vamos considerar um exemplo onde o espaço de endereço tem 64 bytes e o tamanho da página é 16 bytes.

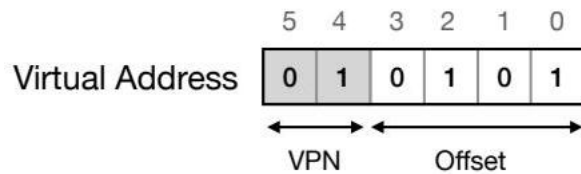


A imagem ilustra como as páginas virtuais (Page 0, Page 1, Page 3) são mapeadas para frames físicos específicos (Frame 3, Frame 7, Frame 2), mostrando a alocação de páginas virtuais para frames físicos disponíveis.

Uma requisição para o endereço virtual 21 bytes (010101 em binário) é feita. O endereço virtual é dividido em:

- **Virtual Page Number (VPN)**: identifica o número da página (bits mais significativos).
- **Offset**: identifica os bytes específicos dentro da página a serem acessados.

No exemplo, o endereço virtual 21 está no 5º byte (0101 em binário) da VPN 1 (01 em binário).

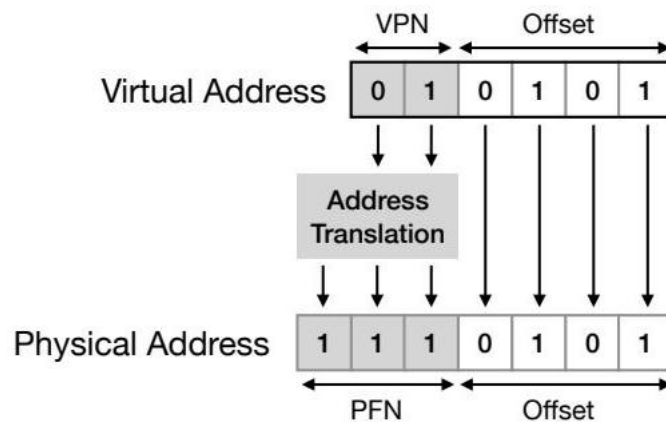


A imagem mostra como um endereço virtual é estruturado, dividido em VPN (Virtual Page Number) e Offset, destacando a função de cada parte no gerenciamento de memória.

Como a Tradução é Feita?

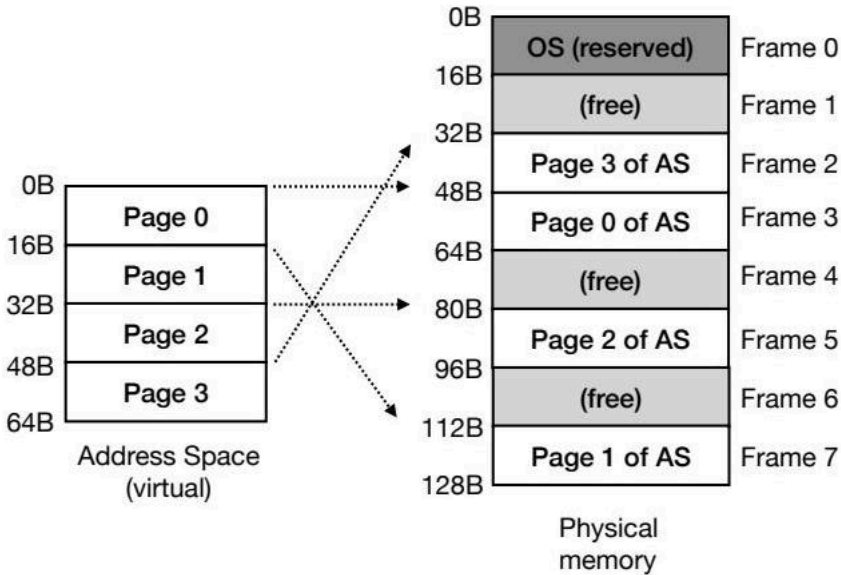
Assumindo que a memória física tem 128 bytes, a tabela de páginas é usada para mapear o VPN para um Physical Frame Number (PFN). VPN 1 é mapeado para PFN 7 (111 em binário). Assim, 1110000 (binário) corresponde a 112 em decimal (endereço do Frame 7!).

O endereço físico (adicionando o offset) é 117 (1110101 em binário).



A imagem demonstra como um endereço virtual (dividido em VPN e Offset) é traduzido para um endereço físico (dividido em PFN e Offset) por meio da tradução de endereços.

PFN também é conhecido como Physical Page Number (PPN).



A imagem ilustra como as páginas no espaço de endereço virtual são mapeadas para frames correspondentes na memória física, mostrando a relação entre páginas virtuais e frames físicos.

Onde as Tabelas de Páginas são Armazenadas?

Para um endereço virtual de 32 bits e páginas de 4KB, precisamos de um offset de 12 bits. Assumindo que cada Page Table Entry (PTE) precisa de 4 bytes (para o PFN e outras informações), 1 milhão de entradas levam a uma tabela de páginas com cerca de 4 MB.

Se 100 processos são carregados na memória, 400 MB de memória são desperdiçados com tabelas de páginas. Para um endereço virtual de 64 bits, teríamos 2^{64} PTEs!

Dado o tamanho das tabelas de páginas, elas devem ser armazenadas na memória física, não no MMU (Memory Management Unit).

O que as Tabelas de Páginas Contêm?

A tabela de páginas é uma estrutura de dados que mapeia VPNs para PFNs. O design mais simples é uma tabela de páginas linear, um array indexado pelo VPN.

Cada elemento do array é um Page Table Entry (PTE) que contém:

- O PFN correspondente.
- Um **bit válido** indicando se a tradução é válida. Espaço não utilizado é marcado como inválido (bit = 0). Se uma página virtual inválida for acessada, uma exceção (trap) é gerada. Esse bit permite suportar espaços de endereço esparsos, sem a necessidade de alocar frames físicos para páginas inválidas!
- **Bits de proteção** indicando se a página pode ser lida, escrita ou executada.
- Um **bit presente** indicando se o conteúdo da página reside na memória ou no disco.
- Um **bit sujo** indicando se a página foi modificada após ser trazida para a memória.
- Um **bit de referência** (ou bit de uso) rastreia se uma página foi acessada.

Tradução de Endereços é Lenta

Para cada acesso à memória feito por um processo, o hardware deve:

1. Saber onde a tabela de páginas do processo está carregada na memória física (através de um registro da CPU).
2. Buscar o PTE correspondente da memória para o processador.
3. Verificar se a requisição é válida, calcular o PFN e concatená-lo com o offset.
4. Acessar o local da memória.

Cada tradução de endereço agora requer dois acessos à memória principal: para a tabela de páginas e para a memória solicitada pelo processo. Isso é mais lento do que a segmentação, onde os limites eram mantidos nos registros da CPU.

Translation Look-aside Buffer (TLB)

Como evitar o acesso extra à memória requerido pelo paging? O SO precisa de ajuda do hardware.

O **Translation Look-aside Buffer (TLB)** é um componente do MMU. É um cache de hardware das traduções de endereço virtual para físico mais populares. Acessar o cache TLB é significativamente mais rápido do que ir para a memória principal!

VPN	PFN
10	101
20	170
2	130

Exemplo simples de entradas TLB

Algoritmo Básico do TLB

1. Extrair o VPN do endereço virtual.
2. Verificar o VPN no TLB.
 - **TLB Hit**: se o TLB contém a tradução.
 - O PFN é extraído, concatenado ao offset para formar o endereço físico, e está feito! (Não precisa consultar a tabela de páginas, economizando um acesso extra à memória)
 - **TLB Miss**: se o TLB não contém a tradução.
 - A tabela de páginas (na memória) deve ser consultada para encontrar a tradução.
 - O TLB deve ser atualizado com essa tradução.
 - O hardware tenta novamente a instrução de acesso à memória.

Como todos os caches, o TLB é construído com a premissa de que a maioria dos acessos resultará em hits. TLB misses resultam no acesso extra à memória que queremos evitar!

Exemplo de TLB

Assumindo um array com 10 elementos (inteiros de 4 bytes), começando no endereço virtual 36, páginas de 16 bytes e ignorando outros acessos à memória, e um TLB vazio:

VPN	PFN	Code
VPN 0		a[0]
VPN 1		
VPN 2		
VPN 3		
VPN 4		
VPN 5		

Tabela de páginas linear

Quando **a[0]** é acessado, ocorre um TLB miss, e a tradução é buscada para o TLB.

VPN	PFN	Code	Miss
VPN 0		a[0]	
VPN 1			
VPN 2			
VPN 3			
VPN 4			
VPN 5			

Tabela de páginas linear

VPN	PFN	Code
2	100	

Exemplo simples de entradas TLB

```
int sum = 0;
for (i=0; i<10; i++) {
    sum += a[i];
}
```

Quando **a[1]** e **a[2]** são acessados, a tradução já está no TLB (hit!). **a[3]** e **a[7]** estão em páginas diferentes, então ocorre um TLB miss quando estes são acessados. Acessos aos elementos restantes do array resultam em hits.

TLB Hit Rate

No exemplo anterior, o padrão é: miss, hit, hit, miss, hit, hit, hit, miss, hit, hit.

Taxa de acerto do TLB: $\frac{7}{10} = 0.7 = 70$

Queremos taxas de acerto próximas a 100% para amortizar o custo de acessar a memória.

Se o array for acessado novamente e as entradas ainda estiverem no TLB, teremos 10 hits!

Localidade Temporal e Espacial

O tamanho da página desempenha um papel importante no número de hits e misses. Um tamanho de página típico de 4KB pode conter vários elementos de arrays densos e alcançar poucos misses.

O TLB está aproveitando duas propriedades dos acessos à memória:

- **Localidade espacial:** Se um programa acessa o endereço x , é provável que acesse endereços próximos a x (ex: um loop iterando sobre um array).
- **Localidade temporal:** Se um programa acessa um item de memória (endereço, dado, instrução), é provável que o acesse novamente em breve (ex: as variáveis i e sum ou o código dentro do loop).

Conteúdo do TLB

Um TLB típico pode ter 32, 64 ou 128 entradas.

O TLB é totalmente associativo, ou seja, uma determinada tradução pode estar em qualquer lugar no TLB e deve ser pesquisada pelo hardware, geralmente em paralelo.

Cada entrada TLB tem:

VPN	PFN	Válido	Prot	ASID
10	100	1	R-X	1
0	-	-	-	-
50	101	1	RW-	2

- O VPN e o PFN correspondente.
- Um bit válido indicando se a tradução de endereço é válida.
- Bits de proteção indicando se a página pode ser lida, escrita ou executada.
- Um bit sujo indicando se a página foi modificada após ser trazida para a memória.
- Um **identificador de espaço de endereço (ASID)**, usado para identificar diferentes processos.

Lidando com TLB Misses e Troca de Contexto

TLB Misses podem ser tratados por:

- **Hardware:** o hardware requer acesso direto às tabelas de páginas na memória (ex: através de um registro base da tabela de páginas).
- **Software:** Em um miss, o hardware levanta uma exceção, salta para o código do SO, o SO trata a exceção, adiciona a tradução ao TLB (instrução de hardware especial) e retorna do trap.

VPN	PFN	Válido	Prot	ASID
10	100	1	R-X	1
0	-	-	-	-
10	101	1	RW-	2
11	30	1	R-X	1
50	30	1	R-X	2

Troca de Contexto

O que acontece quando os processos são trocados (troca de contexto)? As entradas TLB do processo sendo trocado não podem ser usadas pelo novo processo.

- **Opção 1:** Limpar as entradas TLB do processo sendo trocado. No entanto, se o SO agendar o processo novamente em breve, vários TLB misses podem ocorrer.
- **Opção 2:** Usar o campo ASID (similar ao campo PID, mas com menos bits) para indicar o processo ao qual a tradução se refere. O valor ASID é definido pelo SO (em um registro especial).

Compartilhar páginas entre processos é possível tendo diferentes VPNs "apontando" para o mesmo PFN.

Política de Substituição

Quando o TLB está cheio e uma nova entrada deve ser adicionada, qual entrada antiga deve ser substituída (removida)?

Isso requer uma política de substituição. Os principais objetivos devem ser maximizar a taxa de acerto e ser eficiente. Existem vários algoritmos, como FIFO, aleatório, least-recently used (LRU).

O Problema do Tamanho

Ainda temos outro problema para resolver: o tamanho das tabelas de páginas!

Estamos preocupados com o tamanho que as tabelas de páginas ocupam na memória, lembre-se de que essas estruturas precisam ser armazenadas lá!

Lembre-se do nosso exemplo anterior: para um endereço de 32 bits com páginas de 4KB e um PTE de 4 bytes, cada tabela de páginas usa cerca de 4MB. Pior ainda para endereços de 64 bits!

As tabelas de páginas lineares são, portanto, estruturas ineficientes em termos de espaço, especialmente quando as tabelas de páginas de vários processos devem ser carregadas na memória.

Como Reduzir o Tamanho da Tabela de Páginas?

Páginas Maiores

Assumindo o mesmo espaço de endereço de 32 bits, mas com páginas de 16KB:

- Temos um offset de 14 bits ($2^{14} = 16\text{KB}$) e um VPN de 18 bits.
- A tabela de páginas linear teria 262.144 (2^{18}) PTEs.
- Se cada PTE ocupa 4 bytes, cada tabela de páginas ocupa cerca de 1MB.

Uma redução de 4x no tamanho quando comparado a um tamanho de página de 4KB.

Embora páginas grandes resolvam nosso problema de tamanho, elas levam a uma maior fragmentação interna. A maioria dos SOs usa tamanhos de página menores, como 4KB e 8KB, para evitar fragmentação interna severa.

Entradas Inválidas em Tabelas de Páginas Lineares

Muitos processos usam um pequeno espaço de pilha e heap, então o espaço entre os dois não está sendo usado.

Vamos dar uma olhada no espaço de endereço de 16KB com páginas de 1KB abaixo:

- Apenas as páginas 0, 4, 14 e 15 são alocadas na memória física (mapeadas para frames reais).
- Nossa tabela de páginas linear tem vários PTEs marcados como inválidos.

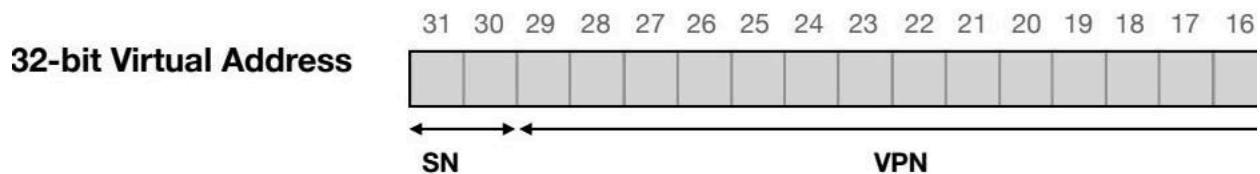
Tabela de páginas para o espaço de endereço de 16KB (linha 0 corresponde a VPN 0, linha 1 a VPN 1 e assim por diante).

Misturando Paging e Segmentação

É possível combinar paging e segmentação em uma abordagem híbrida? Cada segmento (código, heap e pilha) tem sua própria tabela de páginas. Os registros base e limite de cada segmento agora...

Tradução de Endereços com Segmentação e Paging Híbridos

Um endereço virtual de 32 bits com páginas de 4KB é estruturado da seguinte forma: os bits SN (número do segmento) são usados para encontrar os pares base e limites a serem usados. O VPN (número da página virtual) é usado para encontrar o FPN (número do frame físico) na tabela de páginas do segmento, e o offset é adicionado para gerar o endereço físico.



O diagrama acima ilustra como um endereço virtual de 32 bits é dividido em SN, VPN e Offset.

Combinando Paging e Segmentação

Se tivermos uma tabela de páginas por segmento, o espaço da memória física usado pode ser otimizado:

- Se o heap usa apenas as três primeiras páginas (0, 1 e 2), sua tabela de páginas linear requer apenas espaço para 3 PTEs (Page Table Entries).
- O mesmo pode ser feito para os segmentos de código e pilha.
- Evita-se alocar PTEs para o espaço livre entre o heap e a pilha.

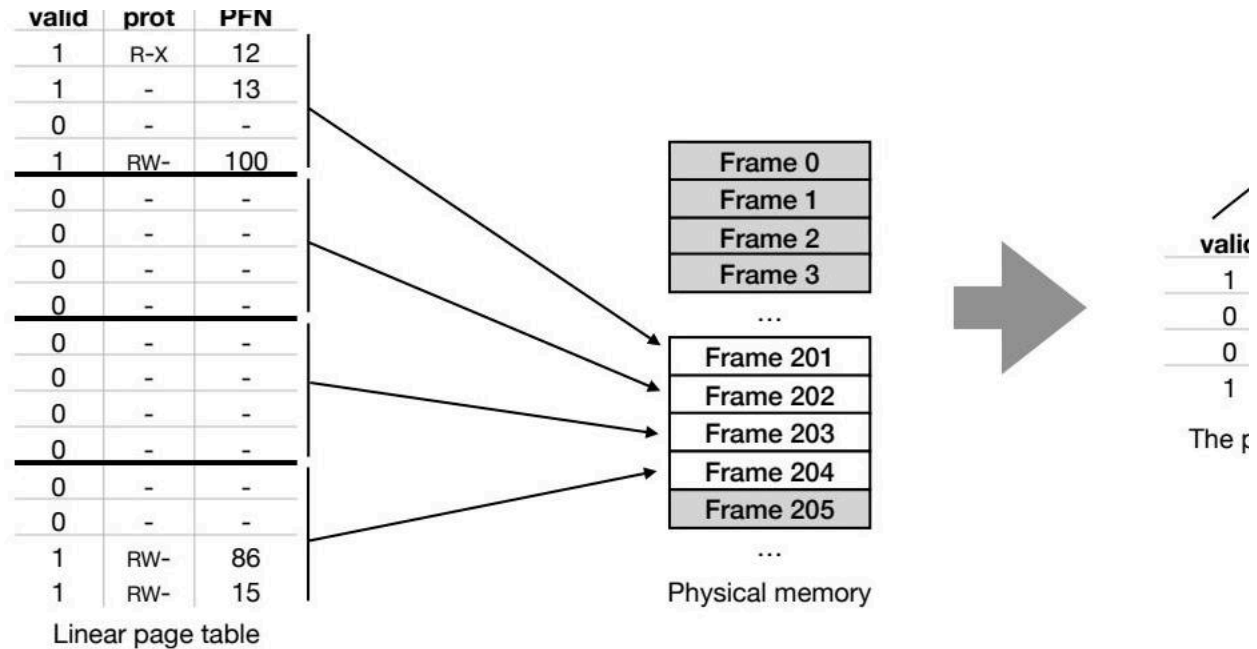
No entanto, esta abordagem híbrida tem desafios:

- Se o heap for alocado esparsamente em várias páginas (por exemplo, páginas 1, 100 e 1500), a tabela de páginas linear deve alocar 1500 PTEs, embora apenas 3 sejam válidas.
- As tabelas de páginas agora têm tamanhos variáveis, e alocar memória para elas leva à fragmentação externa.

Tabelas de Páginas Multi-Nível

Uma forma de evitar PTEs inválidas na memória física é usar tabelas de páginas multi-nível:

- A tabela de páginas linear é dividida em várias unidades do tamanho de uma página.
- Cada unidade é alocada na memória física apenas se contiver pelo menos uma PTE válida.
- Uma nova estrutura, chamada diretório de páginas, é responsável por indexar as unidades de página (ou seja, mantém o PFN da unidade).
- O diretório contém uma entrada de diretório de página (PDE) por unidade indexada. Cada PDE tem um bit válido e um PFN.



A imagem mostra a comparação entre tabelas de páginas lineares e de dois níveis, destacando como as tabelas de dois níveis otimizam a alocação de memória.

Exemplo de Tabela de Páginas de Dois Níveis

Considere um espaço de endereço de 16KB, com páginas de 64 bytes e um tamanho de PTE de 4 bytes:

- Um endereço virtual de 14 bits com 8 bits para VPN e 6 bits para o offset.
- Uma tabela de páginas linear exigiria 2^8 (256) PTEs, usando cerca de 1KB de memória (256 PTEs x 4 bytes).

Para construir uma tabela de páginas de dois níveis:

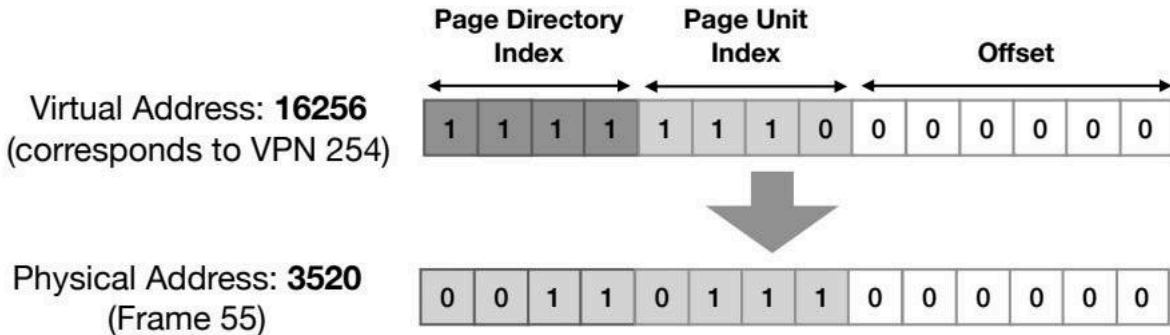
- Dividimos a tabela de páginas linear em unidades de página, cada uma contendo 16 PTEs (64 bytes por página / 4 bytes).
- Para representar o espaço de endereço completo, precisamos de até 16 unidades de página (256 PTEs no total / 16 PTEs por unidade de tabela de páginas).

Este método reduz o uso de memória para 3 frames (1 para o diretório e 2 para as unidades de página), totalizando 192 bytes em vez de 1KB.

Tradução de Endereços em Tabelas de Dois Níveis

A tradução de endereços é semelhante ao que vimos anteriormente. Vamos usar o endereço virtual 16256 como exemplo:

1. **Acesse o diretório de páginas:** O endereço físico é mantido em um registrador base do diretório de páginas. Use os 4 bits superiores do endereço virtual para encontrar o PDE desejado e extrair o PFN da unidade de página. O binário **1111** corresponde à 15ª entrada do diretório, apontando para o PFN 101 (localização da unidade de página).
2. **Acesse a unidade de página:** No frame 101, use os próximos 4 bits do endereço virtual para encontrar a PTE desejada e extrair o PFN do nosso endereço virtual. A 14ª entrada (1110) mapeia para o FPN 55 (00110111).
3. **Adicione o offset:** Neste caso, o offset é 0, então obtemos o endereço físico (o 0º byte do Frame 55!).

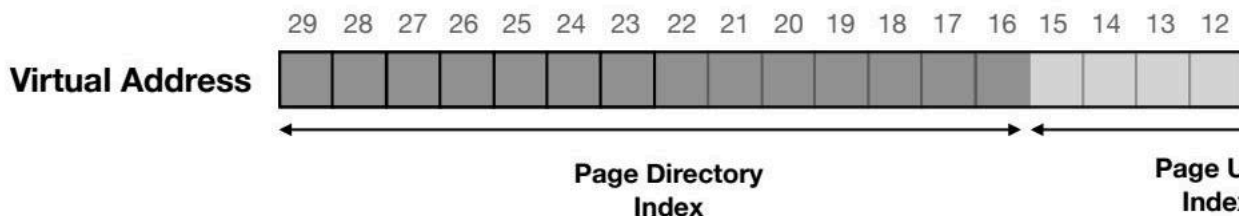


A imagem ilustra a tradução do endereço virtual 16256 para o endereço físico 3520 através do diretório de páginas e da unidade de página.

Mais um Exemplo de Tabelas de Páginas Multi-Nível

Considere um endereço virtual de 30 bits, com um tamanho de página de 512 bytes e uma PTE de 4 bytes:

- 9 bits para o offset e 21 bits para o VPN.
- Cada unidade de página contém 128 PTEs (512 / 4), então precisamos de 7 bits (2^7) para indexar as PTEs.
- O diretório de páginas tem agora 14 bits, contendo $2^{14} = 16384$ entradas, abrangendo 128 páginas.
- Para endereços virtuais de 64 bits, o diretório de páginas pode conter 2^{15} entradas, resultando em um diretório muito grande que requer muita memória, mesmo que muitas PDEs estejam marcadas como inválidas.



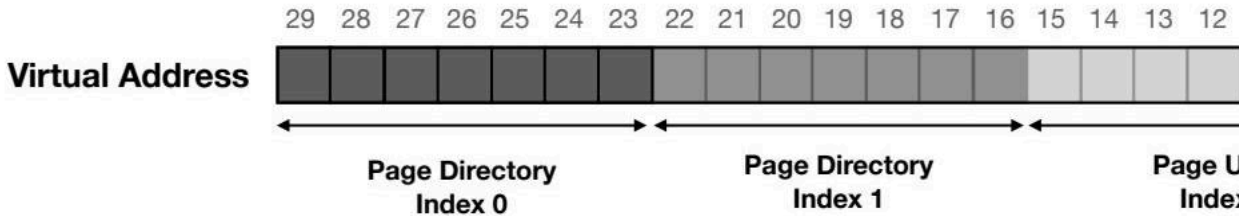
O diagrama detalha a estrutura de um endereço virtual de 30 bits, mostrando seus componentes: índice do diretório de página, índice da unidade de página e offset.

Aumentando os Níveis das Tabelas de Páginas

Uma solução para o problema de diretórios de páginas muito grandes é dividir a árvore em mais de 2 níveis:

- Em um exemplo anterior, se usarmos 3 níveis, o diretório de páginas no nível 0 cabe exatamente em um único frame (contém 128 PDEs).
- Diretórios de nível 1, também cabendo em um único frame, são alocados apenas se contiverem pelo menos uma entrada válida.
- Ter cada diretório e unidade de página cabendo em um único frame é a melhor opção em termos de espaço.
- No entanto, cada vez que ocorre um TLB miss, multiplicamos o número de acessos à memória pelo número de níveis (indireções).

Existe um compromisso entre eficiência de espaço e desempenho. Ter muitos níveis pode não valer a pena!



O diagrama ilustra a divisão de um espaço de endereço virtual de 30 bits em índices de diretório de página (níveis 0 e 1), índice de unidade de página e offset.

Tabelas de Páginas Invertidas

Uma alternativa para economizar espaço de memória é ter uma tabela de páginas compartilhada entre todos os processos:

- Cada frame (PFN) tem uma entrada na tabela mapeando para:
 - O VPN
 - O identificador do processo (PID)

PFN	VPN	PID
0	10	1
1	11	1
2	12	1
4	10	0
5	20	2
6	30	0
7	13	1

Encontrar a entrada correta requer uma varredura linear através da lista. Uma tabela hash pode ser construída sobre a estrutura base para acelerar as buscas.

Hierarquia de Memória

Estudamos diferentes maneiras de reduzir a memória usada por espaços de endereço e tabelas de páginas. Mesmo com as técnicas anteriores, se muitos programas estiverem em execução, podemos querer reduzir ainda mais o uso de memória sem comprometer:

- **Conveniência:** Programadores não devem se preocupar em escrever programas cujas estruturas de dados devem caber na memória física.
- **Desempenho:** Ainda queremos que os programas sejam executados rapidamente (ou seja, acessar essas estruturas de dados de forma eficiente).

E se o SO salvar (trocar) porções do espaço de endereço (por exemplo, páginas) que não estão em grande demanda para outro local com maior capacidade, como o dispositivo de disco?

- A troca também pode ser aplicada para porções de tabelas de páginas, reduzindo ainda mais a memória usada por elas!
- Dispositivos de disco são mais lentos que a memória física e estão na parte inferior da hierarquia de memória.

O Espaço de Troca (Swap)

Se as páginas serão trocadas da memória para o disco, devemos reservar algum espaço neste último. Este local é chamado de espaço de troca (swap).

O tamanho alocado para o espaço de troca é importante porque, quando combinado com o espaço de memória física, dita o número total de páginas utilizáveis no sistema.

- Processos 0, 1 e 2 estão em execução com algumas páginas na memória e outras no espaço de troca.
- O Processo 3 tem todas as páginas no disco (espaço de troca); portanto, provavelmente, não está em execução.

Proc 0 [VPN 0]	Proc 1 [VPN 1]	Proc 1 [VPN 3]	Proc 2 [VPN 0]
PFN 0	PFN 1	PFN 2	PFN 3
Physical Memory			
Proc 0 [VPN 1]	Proc 0 [VPN 2]	Proc 1 [VPN 0]	[Livre]
Block 0	Block 1	Block 2	Block 3
Swap Space			

Bit Presente e Falha de Página

Quando o SO ou hardware (dependendo da abordagem) está procurando na tabela de páginas por uma tradução, a entrada PTE deve indicar se a página está na memória física ou no disco.

- A PTE deve ter um bit presente definido como 1 se a página estiver na memória ou 0 se estiver no disco.

O ato de acessar uma página que não está na memória é chamado de **page fault** (falha de página).

Sob uma falha de página, o SO deve ser invocado para executar um manipulador de falha de página (para trazer a página do disco para a memória). O PFN (na PTE) pode ser usado para manter o endereço do disco!

Address!

Disk address! →

Page table

PFN	valid	prot	present	dirty
1000	1	R-X	0	0
-	0	-	-	-
-	0	-	-	-
4	1	RW-	1	1
...				

A imagem mostra uma tabela de páginas onde o bit "present" (presente) indica se a página está na memória ou no disco, com o PFN apontando para o endereço no disco.

Manipulador de Falha de Página

Quando invocado, o código do manipulador de falha de página (no SO) deve:

- Encontrar um frame livre na memória para a página (e se não houver frames disponíveis?).
- Ler a página do disco (requisição de I/O) e colocá-la na memória.
- Quando a I/O estiver completa, a PTE correspondente deve ser atualizada (o bit presente definido como 1 e o PFN atualizado para o número do frame de memória física).
- Tentar novamente a instrução de acesso à memória.
- Esta nova tentativa pode gerar um TLB miss que deve ser tratado adicionando a nova tradução ao TLB (uma otimização é atualizar o TLB ao tratar a falha de página para evitar este passo extra).

Enquanto a página está sendo lida do disco, o estado do processo é bloqueado, e o SO pode executar outro programa. Isso otimiza o uso do hardware (sobreposição de I/O e tempo de CPU).

E se a Memória Estiver Cheia?

Se a memória estiver cheia (ou seja, não houver frames disponíveis), o SO pode precisar primeiro remover (trocar para o disco) uma ou mais páginas para liberar espaço para a(s) nova(s) página(s). Só então pode ler a nova página para a memória física (para um frame).

O processo de escolher quais páginas substituir (remover) é conhecido como **page-replacement policy** (política de substituição de páginas).

Esperar que a memória fique cheia para substituir páginas é uma má decisão. O manipulador de falha de página precisa esperar que as páginas sejam liberadas para prosseguir com seu trabalho. E se as páginas forem liberadas em segundo plano?

Swap Daemon

Para lidar com a falta de memória, um daemon de troca pode ser utilizado:

- Quando? Quando a quantidade de frames disponíveis está abaixo de um dado número (low watermark - LW).
- Quantas páginas devem ser liberadas? Tantas quanto necessário para atingir outro número (high watermark - HW).

Ter um daemon muda o fluxo de trabalho do manipulador de falha e permite otimizações interessantes:

- O manipulador de falha de página não é mais responsável por substituir páginas. Ele apenas verifica se há frames livres e, caso contrário, informa o daemon e aguarda uma notificação dele.
- O daemon pode agrupar várias páginas para remover em uma única requisição de flush para o disco.

Todos os mecanismos descritos são feitos de forma transparente para os processos. Cada processo acessa sua própria memória privada sem se preocupar se a página está residindo no disco ou na memória. O SO faz todo o trabalho!

Portanto, temos nossa abstração de memória virtual que se estende além da memória física!

Desafio: Políticas de Substituição de Páginas

Com a memória virtual, a memória física pode ser vista como um cache de páginas no sistema (as outras páginas são armazenadas no disco!). O objetivo do SO é minimizar o número de cache misses (ou seja, o número de falhas de página!). Alternativamente, podemos dizer que o SO quer maximizar os cache hits (ou seja, o número de vezes que uma página é encontrada na memória!).

Vamos usar como nossa métrica (para avaliar nossas políticas) o Tempo Médio de Acesso à Memória (AMAT):

- TM: custo de acessar a memória
- TD: custo de acessar o disco
- PMISS: probabilidade de não encontrar os dados na memória (um miss). PMISS varia de 0.0 a 1.0

$$AMAT = TM + (PMISS \times TD)$$

Intuição: você sempre paga o custo de acessar a memória ao buscar uma página, mas quando você tem um miss, você também paga o custo de buscar dados do disco.

Exemplo de AMAT

Suponha 10 páginas virtuais (páginas 0 a 9), todas na memória exceto a página 3. Se um programa acessa cada uma dessas páginas uma vez:

- O seguinte comportamento de cache acontecerá: hit hit hit miss hit hit hit hit hit hit (PMiss = 0.1)
- O custo de acessar o disco é de 10 milissegundos, enquanto acessar a memória leva 100 nanossegundos (0.0001 ms)

$$AMAT = 0.0001ms + (0.1 \times 10)ms = 1.0001ms$$

Se PMISS é 0.001 (ou seja, 99.9% de taxa de hit):

$$AMAT = 0.0001ms + (0.001 \times 10)ms = 0.0101ms = 10.1\mu s$$

Como o custo de acessar o disco é muito maior do que acessar a memória, é preciso ter taxas de miss muito baixas (ou seja, é preciso de boas políticas de substituição de páginas!).

Política Ótima

Existe uma política de substituição de páginas ótima (ou seja, levando ao menor número de misses no geral)? Sim, Belady propôs há muitos anos (originalmente chamada MIN).

Acesso	Hit/Miss?	Evict	Estado do Cache Resultante
0	MISS		0
1	MISS		0, 1
2	MISS		0, 1, 2
0	HIT		0, 1, 2
1	HIT		0, 1, 2
3	MISS	2	0, 1, 3
0	HIT		0, 1, 3
3	HIT		0, 1, 3
1	HIT		0, 1, 3
2	MISS	3	0, 1, 2
1	HIT		0, 1, 2

Estratégia: substitua a página que será acessada mais distante no futuro!

Suponha um programa que acessa o seguinte fluxo de páginas: 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1. O cache tem 3 frames disponíveis e começa com um estado vazio, ou seja, a primeira vez que uma página diferente é solicitada, resulta em um miss (conhecido como cold-start miss).

Taxa de hit: (75%, excluindo cold-start misses) $\text{hits} / \text{accesses} = 6 / 11 = 0.545 = 54.5\%$

Problema: o futuro geralmente não é conhecido e, portanto, não podemos construir esta política para um SO de propósito geral. Mas podemos usar a política ótima para fins de comparação!

FIFO (first-in, first-out)

Sistemas antigos empregavam políticas de substituição muito simples como FIFO.

Acesso	Hit/Miss?	Evict	Estado do Cache Resultante
0	MISS		0
1	MISS		0, 1
2	MISS		0, 1, 2
0	HIT		0, 1, 2
1	HIT		0, 1, 2
3	MISS	0	1, 2, 3
0	MISS	1	2, 3, 0
3	HIT		2, 3, 0
1	MISS	2	3, 0, 1
2	MISS	3	0, 1, 2
1	HIT		0, 1, 2

Estratégia: Substitua a página que está há mais tempo na memória. Fácil de implementar! Adicione páginas a uma fila quando elas entram no sistema e substitua a que está no final da fila.

Assumindo o mesmo fluxo de acessos de páginas: Taxa de hit: (50%, excluindo cold-start misses) $\text{hits} / \text{accesses} = 4 / 11 = 0.364 = 36.4\%$

Anomalia de Belady (observação lateral)

E se tentarmos FIFO com o seguinte fluxo de acessos de página: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Resultado estranho!

- Com 3 frames disponíveis - 3 hits!
- Com 4 frames disponíveis - 2 hits!

Conclusão: com FIFO, aumentar o tamanho do cache pode não significar uma taxa de hit melhor.

Aleatório

Acesso	Hit/Miss?	Evict	Estado do Cache Resultante
0	MISS		0
1	MISS		0, 1
2	MISS		0, 1, 2
0	HIT		0, 1, 2
1	HIT		0, 1, 2
3	MISS	0	1, 2, 3
0	MISS	1	2, 3, 0
3	HIT		2, 3, 0
1	MISS	3	2, 0, 1
2	HIT		2, 0, 1
1	HIT		2, 0, 1

Estratégia: Substitua uma página aleatória. Muito fácil de implementar, como FIFO. A razão de hits e misses depende de fazer escolhas de sorte ou azar!

Voltando ao nosso fluxo original de acessos de página: Taxa de hit: (62.5%, excluindo cold-start misses) $\text{hits} / \text{accesses} = 5 / 11 = 0.455 = 45.5\%$

Uma estratégia aleatória exibirá comportamentos diferentes se alguém repetir o mesmo fluxo de acessos várias vezes.

Usando Frequência e Recência

FIFO e Random são simples de implementar, mas não olham para a importância das páginas. Como fizemos para o escalonamento de processos, vamos olhar para o passado para tomar melhores decisões no futuro!

- **Frequência:** se uma página foi acessada muitas vezes no passado, talvez valha a pena não substituí-la.
- **Recência:** se uma página foi acessada recentemente, talvez ela seja acessada novamente em um futuro próximo.

Políticas que olham para essas métricas são baseadas, novamente, no princípio da localidade:

- **Localidade espacial:** se uma página P for acessada, as páginas ao redor dela (digamos P - 1 ou P + 1) também serão acessadas (por exemplo, iterando através de um array).
- **Localidade temporal:** páginas acessadas no passado recente são propensas a serem acessadas novamente no futuro próximo (por exemplo, acessando as mesmas variáveis dentro de um loop).

A localidade pode não estar presente em todos os programas.

LRU (Least Recently Used)

Duas políticas principais surgem:

- **Least Frequently Used (LFU):** substitua a página menos frequentemente usada.
- **Least Recently Used (LRU):** substitua a página menos recentemente usada.

As políticas opostas também existem: Most Frequently Used (MFU) e Most Recently Used (MRU). Na maioria dos casos (nem todos), essas políticas não funcionam bem, pois não abraçam a localidade.

Vamos olhar para LRU para nosso exemplo em execução.

Acesso	Hit/Miss?	Evict	Estado do Cache Resultante
0	MISS		0
1	MISS		0, 1
2	MISS		0, 1, 2
0	HIT		1, 2, 0
1	HIT		2, 0, 1
3	MISS	2	0, 1, 3
0	HIT		1, 3, 0
3	HIT		1, 0, 3
1	HIT		0, 3, 1
2	MISS	0	3, 1, 2
1	HIT		3, 2, 1

Assumindo o mesmo fluxo de acessos de páginas: Taxa de hit: (75%, excluindo cold-start misses) $\text{hits} / \text{accesses} = 6 / 11 = 0.545 = 54.5\%$

Implementando LRU

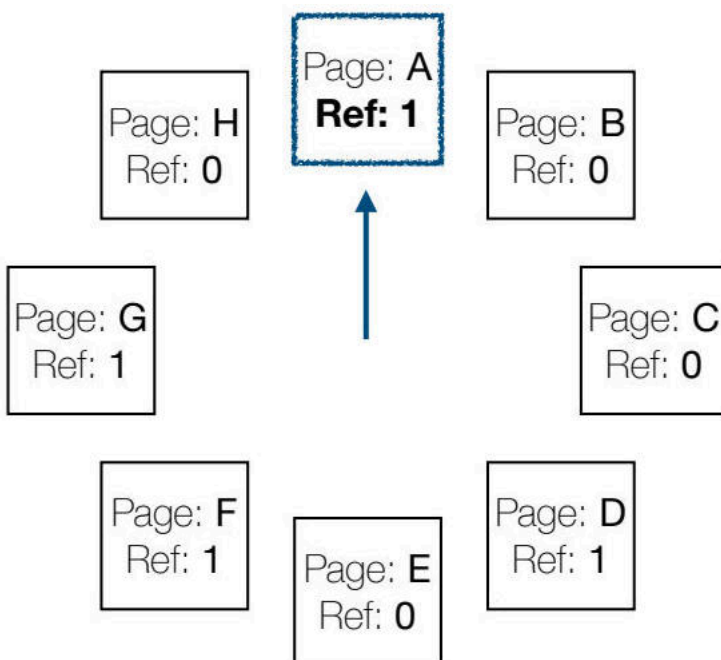
LRU fornece os melhores resultados até agora. No entanto, implementá-lo perfeitamente é desafiador:

- Com alguma ajuda do hardware, os timestamps dos acessos de página poderiam ser salvos, por exemplo, em tabelas de páginas. Então, o SO precisaria escanear a lista completa de timestamps para todas as tabelas de páginas para remover a página menos recentemente usada.
- Outra opção seria ter uma fila como em FIFO, mas quando uma página é acessada, deve-se procurá-la na lista e promovê-la para a cabeça da lista.

Ambas as abordagens exigem muito trabalho do SO e atrasariam a remoção de páginas (o SO deve ser eficiente!). Imagine escanear milhões de páginas...

Aproximando LRU

Sistemas modernos aproximam o comportamento de LRU.

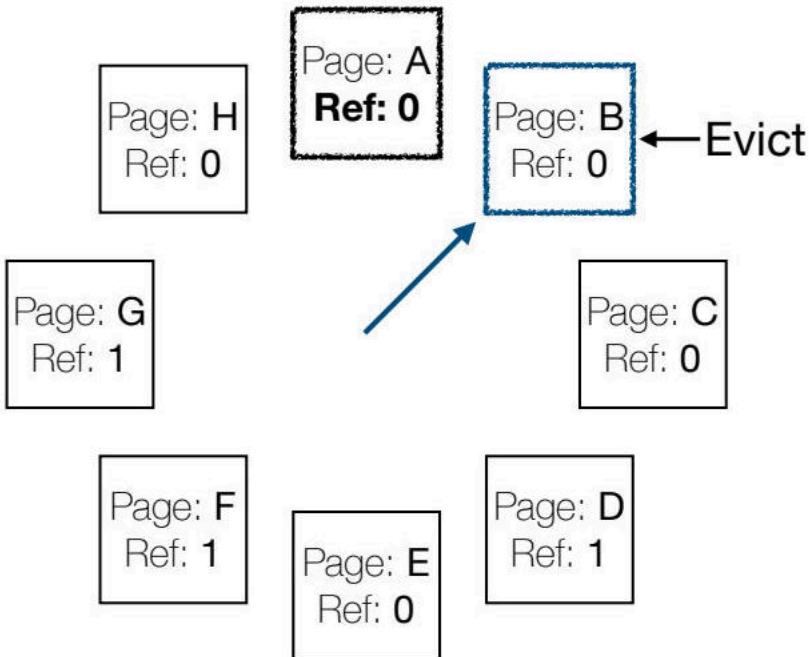


O diagrama acima ilustra um conceito relacionado à referência ou mapeamento de páginas, essencial para entender a aproximação de LRU.

Um bit de referência (também chamado de bit de uso) na PTE é definido pelo hardware para 1 quando a página correspondente é acessada.

O Algoritmo do Relógio (aka Segunda Chance)

- Imagine todas as páginas do sistema organizadas em uma lista circular.
- Um ponteiro de relógio aponta para uma página.
- Quando uma página deve ser substituída, o SO olha para a página apontada pelo ponteiro do relógio.
 - Se o bit de referência é 1, a página foi usada recentemente, então o bit é definido para 0 pelo SO, e nosso algoritmo de relógio avança para a próxima página (move o ponteiro do relógio).

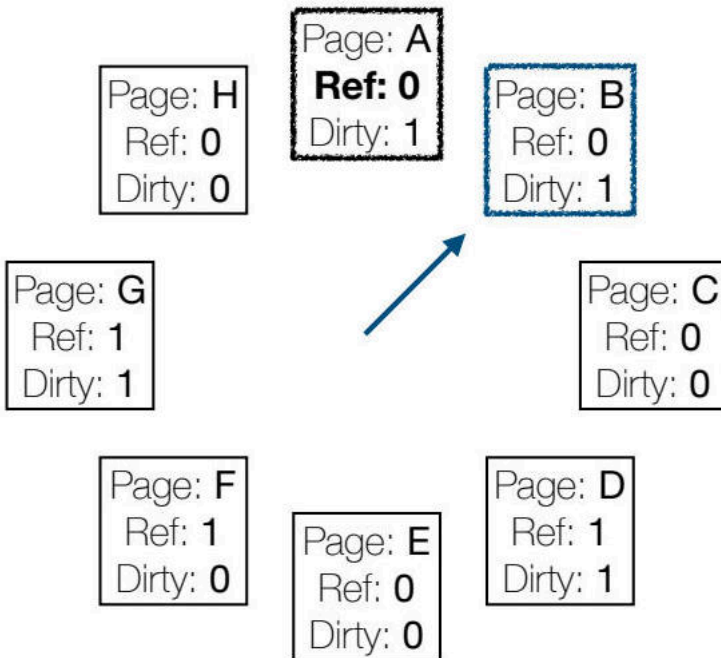


O diagrama acima ilustra o processo de remoção de página em um sistema de computador, com cada página tendo um bit de referência.

Aproximando as Políticas de Substituição de Páginas LRU

Uma pequena melhoria no algoritmo anterior considera se o conteúdo da página foi modificado após trazer (trocar) a página para a memória.

Observe a imagem abaixo, que mostra um diagrama de oito páginas (A a H), cada uma representada por uma caixa contendo informações sobre a página. As caixas estão dispostas em duas fileiras de quatro, com a página A no centro superior e uma seta apontando dela para a página B. Cada caixa exibe o nome da página, um valor de referência (Ref) e um valor de "dirty" (sujo), com as páginas A, B, D e G tendo um valor "dirty" de 1, indicando que foram modificadas. O valor de referência provavelmente indica se a página foi acessada recentemente.



- **Bit Dirty:** Um bit "dirty" é configurado para 1 pelo hardware quando a página é modificada.

Algumas páginas serão trocadas dentro e fora da memória várias vezes. Se houver espaço suficiente na área de troca, o SO pode deixar o conteúdo das páginas lá, mesmo quando estas são trocadas de volta para a memória. Com tal truque:

- A remoção de uma página não modificada (não suja) não requer uma operação extra de E/S de disco (ou seja, o conteúdo mais recente da página já está na área de troca).
- A remoção de uma página suja requer a escrita do conteúdo modificado de volta no disco (ou seja, atualizar o conteúdo da página na área de troca).

Algoritmo de Clock (Aprimorado):

1. Primeiro, tente remover páginas que não foram acessadas recentemente ou modificadas.
2. Em seguida, escolha páginas que não foram acessadas recentemente, mas que estão sujas.

Comparação das Políticas de Substituição de Páginas

- **FIFO**, **Random** e **LRU** têm desempenho semelhante quando os programas não exibem nenhuma propriedade de localidade. O principal fator para melhorar a taxa de acertos é o tamanho do cache.
- Quando os programas exibem alta localidade, **LRU** tem um desempenho melhor. Por exemplo, quando uma grande porcentagem de solicitações (por exemplo, 80%) é repetidamente feita para um pequeno conjunto de páginas (por exemplo, 20%) que cabem no espaço do cache.
- Quando os programas escaneiam sequencialmente grandes porções de memória (por exemplo, um array muito grande) e o tamanho do cache é inferior às páginas que estão sendo escaneadas, então **FIFO** e **LRU** têm um desempenho ruim (pior cenário para essas estratégias). **Random** tem um desempenho um pouco melhor para lidar melhor com essas cargas de trabalho de varredura (por exemplo, o algoritmo ARC).

Para mais detalhes sobre essas diferentes cargas de trabalho, consulte a Seção 22.6 de Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.

Prefetching e Clustering: Outras Técnicas

Além de substituir páginas, o gerenciador de memória virtual também emprega outras políticas e otimizações:

- Para muitas páginas, o SO aplica **paginação por demanda**, o que significa que as páginas em disco só são buscadas na memória quando acessadas.
- No entanto, em outros casos, o SO pode adivinhar que uma página está prestes a ser usada e pode pré-busca-la do disco para a memória (por exemplo, útil para páginas que contêm o código de programas).
- Outra política que discutimos antes consiste em agrupar (clustering) várias páginas para serem escritas no disco de uma só vez.

Fenômeno Thrashing

Às vezes, o conjunto de processos em execução excede a memória disponível e não há boas páginas candidatas para remover.

Pior, se o SO decidir remover as páginas dos processos que "imediatamente" serão solicitadas em seguida (por exemplo, o agendador decide executar esses processos novamente), isso levará a mais faltas de página, o que levará a outras faltas de página, ...

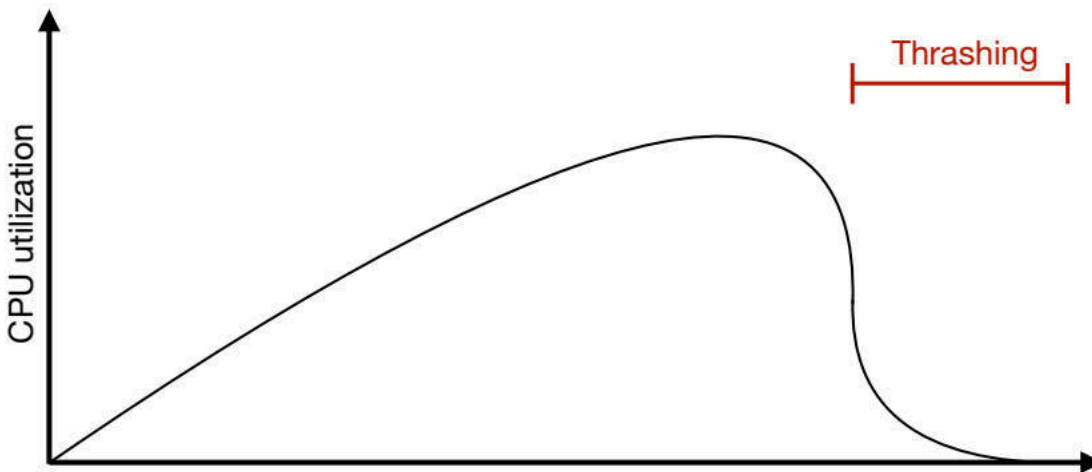
Este fenômeno, conhecido como **thrashing**, faz com que o SO gaste a maior parte do seu tempo lidando com faltas de página e trocando páginas em vez de executar processos!

Scheduler Thrashing

Na verdade, se o agendador de processos do SO for ingênuo, pode levar ao seguinte cenário:

- À medida que o sistema está em thrashing, a utilização da CPU cai porque mais solicitações de E/S de disco estão sendo feitas (ou seja, tratamento de faltas de página).
- O agendador vê um baixo uso da CPU e decide executar mais processos, levando a um fenômeno de thrashing mais acentuado.
- Isso significa que olhar apenas para a CPU é uma má ideia. O agendador também deve olhar para a taxa de faltas de página, por exemplo!

A imagem abaixo mostra um gráfico de linhas ilustrando a relação entre a utilização da CPU e uma variável não especificada no eixo x. O gráfico apresenta uma linha curva que inicialmente aumenta, atinge um pico e depois declina acentuadamente. Um rótulo vermelho no canto superior direito destaca uma seção da curva, rotulada como "Thrashing". O eixo y é rotulado como "Utilização da CPU", enquanto o eixo x não tem rótulo. Este gráfico exemplifica como o thrashing afeta a utilização da CPU.



Solução para Thrashing

Sistemas antigos tentariam estimar o conjunto de trabalho (ou seja, o número de páginas ativamente usadas) de todos os processos e executar um subconjunto destes que não excederia a memória disponível.

Esta abordagem, conhecida como **controle de admissão**, é baseada na ideia de que fazer menos trabalho e bem é melhor do que fazer tudo de uma vez, mas mal.

Sistemas modernos normalmente executam um eliminador de falta de memória que escolhe e mata um processo intensivo em memória. Isso é um pouco problemático se o processo for o servidor X (deixando aplicativos que usam o display inutilizáveis...).

Mais Informações

- Capítulos 18 a 22 - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.
- Avi Silberschatz, Peter Baer Galvin, Greg Gagne. Operating System Concepts (10. ed). John Wiley & Sons, 2018.
- Quer saber ainda mais sobre sistemas completos de memória virtual?
- Capítulo 23 - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.